

Typst Template Infrastructure

Comprehensive Typst template system for WAFT, providing access to official Typst templates and community templates with a unified API.

Overview

The Typst infrastructure provides a comprehensive, production-ready system for generating PDF documents using Typst, a modern typesetting system designed as a LaTeX alternative. This infrastructure seamlessly integrates with WAFT's existing template systems while providing enhanced security, performance, and developer experience.

Key Features

- **TypstCompiler:** Secure, hardened compiler for Typst documents with comprehensive error handling
- **TypstTemplateRegistry:** Auto-discovery system for template wrappers with metadata extraction
- **10+ Templates:** Official Typst templates and community templates covering academic papers, books, letters, and more
- **Unified API:** Consistent interface across all templates for easy integration
- **Security First:** Built-in protection against path traversal, command injection, and resource exhaustion
- **Production Ready:** Comprehensive error handling, logging, and timeout management
- **Developer Friendly:** Auto-discovery, clear error messages, and extensive documentation

Why Typst?

Typst offers several advantages over traditional LaTeX:

- **Faster Compilation:** Single-pass compilation typically completes in under a second
- **Modern Syntax:** Clean, readable syntax with built-in scripting capabilities
- **Better Error Messages:** Clear, actionable error messages with precise location information
- **Package System:** Easy-to-use package management via Typst Universe
- **Active Development:** Regular updates and active community support
- **Web-Based Editor:** Optional web-based editing experience at typst.app

Use Cases

The Typst infrastructure is ideal for:

- **Academic Publishing:** Research papers, preprints, conference submissions
- **Technical Documentation:** API docs, user manuals, technical reports
- **Business Documents:** Letters, reports, newsletters, proposals
- **Creative Projects:** Books, zines, campaign materials, game content
- **Automated Generation:** Programmatic PDF generation from structured data
- **Multi-Format Publishing:** Generate PDFs from the same source as web content

Installation

Prerequisites

1. **Typst CLI** (required)
2. Minimum version: 0.10.0
3. Installation options: ````bash # Using Cargo cargo install typst-cli`

Or download from <https://typst.com> ``

4. Python Dependencies

5. Standard library only (no additional packages required)

6. Uses: `subprocess`, `tempfile`, `pathlib`, `shutil`

Verify Installation

```
from src.waft.templates.typst import TypstCompiler

# This will raise RuntimeError if Typst is not available
compiler = TypstCompiler()
print("✅ Typst is available!")
```

Quick Start

Basic Usage

The simplest way to generate a PDF is to compile Typst content directly:

```
from pathlib import Path
from src.waft.templates.typst import TypstCompiler

# Create compiler with default settings
compiler = TypstCompiler()

# Compile Typst content to PDF
typst_content = """
#set page(margin: 2cm)

= Hello World

This is a simple Typst document.

== Introduction

Typst is a modern typesetting system that makes it easy to create beautiful documents.

== Features

- Fast compilation
- Clean syntax
- Powerful features
"""

pdf_path = compiler.compile(
    typst_content=typst_content,
    output_path=Path("output.pdf")
)

print(f"Generated: {pdf_path}")
```

Compiling from Files

If you have existing `.typ` files, you can compile them directly:

```
from pathlib import Path
from src.waft.templates.typst import TypstCompiler

compiler = TypstCompiler()

# Compile a Typst file
pdf_path = compiler.compile_file(
    typ_file=Path("document.typ"),
    output_path=Path("document.pdf")
)

print(f"Generated: {pdf_path}")
```

Custom Configuration

You can customize the compiler behavior:

```
from pathlib import Path
from src.waft.templates.typst import TypstCompiler

# Create compiler with custom settings
compiler = TypstCompiler(
    timeout=120,                      # 2 minute timeout for large documents
    max_content_size=20 * 1024 * 1024 # 20MB content limit
)

# Use for large or complex documents
pdf_path = compiler.compile(
    typst_content=large_content,
    output_path=Path("large_document.pdf")
)
```

Using Templates

```
from pathlib import Path
from src.waft.templates.typst import get_typst_registry

# Get registry
registry = get_typst_registry()

# List all templates
templates = registry.list_templates()
for template in templates:
    print(f"- {template.name} ({template.category})")

# Get a specific template
template = registry.get_template("Flow Way")
generate_func = registry.get_generate_function("Flow Way")

# Generate PDF
pdf_path = generate_func(
    title="My Document",
    content="# Introduction\n\nContent here...",
    output_path=Path("output.pdf"),
    authors=["John Doe"],
    toc=True
)
```

Available Templates

Document Templates

Academic Papers

Arkheion (`preprint`) - arXiv-style academic paper template - Features:

Multiple authors, abstract, keywords, bibliography, appendices - Usage: `python`

```
generate_func = registry.get_generate_function("arkheion")
pdf_path = generate_func( title="My Research Paper",
content="# Introduction\n\n...",
output_path=Path("paper.pdf"), authors=[ { "name": "John
Doe", "email": "john@university.edu", "affiliation":
"University Name", "orcid": "0000-0000-0000-0000" } ],
abstract="This paper presents...", keywords=["machine
learning", "typst"], date="January 19, 2026" )
```

Charged IEEE (`paper`) - IEEE conference paper template - Features: Two-column layout, tight spacing, numeric citations - Usage: `python`

```
generate_func = registry.get_generate_function("Charged
IEEE") pdf_path = generate_func( title="IEEE Paper Title",
content="# Introduction\n\n...",
output_path=Path("ieee.pdf"), authors=[ { "name": "Author
Name", "department": "Department", "organization":
"Organization", "location": "Location", "email":
"email@example.com" } ], abstract="Abstract text...",
index_terms=["Term 1", "Term 2"], bibliography="refs.bib" )
```

Unequivocal AMS (`paper`) - American Mathematical Society paper template -

Features: Single-column, theorem/proof functions, bibliography - Usage: `python`

```
generate_func = registry.get_generate_function("Unequivocal
AMS") pdf_path = generate_func( title="Mathematical Paper",
content="# Introduction\n\n...", output_path=Path("ams.pdf"),
authors=[ { "name": "Mathematician", "department":
"Mathematics", "organization": "University", "email":
```

```
"math@university.edu" } ], abstract="Abstract...",  
bibliography="refs.bib" )
```

Books & Reports

Wonderous Book (`book`) - Fiction book template - Features: Title page, TOC, chapter template, dedication - Usage:

```
python generate_func =  
registry.get_generate_function("Wonderous Book") pdf_path =  
generate_func( title="My Novel", content="# Chapter  
1\n\n...", output_path=Path("book.pdf"), author="Author  
Name", dedication="For my readers",  
publishing_info="Publisher info..." )
```

Flow Way (`report`) - Modern document/report template - Features: Clean design, customizable colors, TOC - Usage:

```
python generate_func =  
registry.get_generate_function("Flow Way") pdf_path =  
generate_func( title="My Report", content="#  
Introduction\n\n...", output_path=Path("report.pdf"),  
authors=["John Doe", "Jane Smith"], affiliation="Company  
Name", year=2026, toc=True, toc_depth=3 )
```

Letters & Newsletters

Appreciated Letter (`letter`) - Business/personal letter template - Features:

Sender/recipient addresses, date, subject - Usage:

```
python generate_func =  
registry.get_generate_function("Appreciated Letter") pdf_path  
= generate_func( content="Dear John,\n\nLetter content...  
\n\nBest regards,", output_path=Path("letter.pdf"),  
sender="Jane Smith\n123 Main St\nCity, State",  
recipient="John Doe\n456 Oak Ave\nCity, State", date="January  
19, 2026", subject="Subject Line", name="Jane Smith" )
```

Dashing Dept News (`newsletter`) - Departmental newsletter template -

Features: Hero image, main column, sidebar articles - Usage:

```
python  
generate_func = registry.get_generate_function("Dashing Dept  
News") pdf_path = generate_func( title="Department
```

```
Newsletter", content="# Main Article\n\n...",  
output_path=Path("newsletter.pdf"), edition="March 2026",  
hero_image={ "image": "cover.jpg", "caption": "Award-winning  
research" }, publication_info="Publication details..." )
```

Game Templates

Badformer (`game`) - Retro platformer game (interactive)

Cereal Words (`game`) - Word puzzle game (interactive)

Icicle (`game`) - Christmas puzzle game (interactive)

Note: Game templates are best viewed with `typst watch` for interactivity.

Future Integration Templates

The following templates are planned for integration:

LaPreprint Template (Academic Preprints)

Repository: <https://github.com/myst-templates/lapreprint-typst.git>

- **Purpose:** arXiv-style academic paper template
- **Features:** Color schemes, author/ORCID support, multiple abstracts, bibliography styles
- **Status:** Planned
- **Integration:** Will be available as `preprint` template in registry

D&D 5e Template (RPG Content - General)

Repository: <https://github.com/coljac/typst-dnd5e.git>

- **Purpose:** General D&D 5e content formatting
- **Features:** Stat blocks, spell formatting, breakout boxes, tables
- **Package:** Available as `@preview/dragonling:0.2.0` on Typst Universe
- **Status:** Planned

- **Integration:** Will integrate with existing WAFT D&D functionality (`dnd_scenario.py`, `dnd5e_latex.py`)
- **Note:** Different from wenyuan-campaign (general content vs. campaign documents)

Wenyuan Campaign Template (D&D 5e Campaigns)

Repository: <https://github.com/yanwenywan/typst-packages.git> (campaign branch)

- **Purpose:** D&D 5e campaign/adventure documents
- **Features:** Campaign documents, statblocks, D&D items, multi-column layouts
- **Package:** Available as `@preview/wenyuan-campaign:0.1.2` on Typst Universe
- **Status:** Planned
- **Note:** Requires custom fonts (TeX Gyre Bonum, Scaly Sans, etc.) - font path handling needed
- **Integration:** Will complement typst-dnd5e for campaign-specific documents

Reference Resources

The following resources have been collected for context and future integration:

Typst Templates (To Integrate)

1. **lapreprint-typst** - Academic preprint template
2. **typst-dnd5e** - General D&D 5e content template
3. **wenyuan-campaign** - D&D 5e campaign document template

Data Generation Tools

1. **statblock5e-creator** - Web app for creating creature statblocks (exports JSON)
2. URL: <https://github.com/Frumple/statblock5e-creator>

3. Use case: Generate data for Typst D&D 5e templates
4. **Aurora-Homebrew-GUI** - Python GUI for creating homebrew spells (exports XML)
5. URL: <https://github.com/ERRORCODE509/Aurora-Homebrew-GUI>
6. Use case: Reference for spell data structures

Data Reference Tools

1. **weavelore** - Vue.js spell browser (spell data reference)
2. URL: <https://github.com/bgior/weavelore>
3. **dnd-spellbook** - Next.js spell manager (spell data management)
4. URL: <https://github.com/andreafra/dnd-spellbook>
5. **dnd5e-companion** - Reference tables and resources
6. URL: <https://github.com/acodcha/dnd5e-companion>

Character Creation Tools

1. **orcpub** - Full character builder (Clojure/ClojureScript)
2. URL: <https://github.com/Orcpub/orcpub>
3. Use case: Architecture reference for character building systems
4. **DND-Randomised-Character-Creator** - Random character generator (JavaScript)
 - URL: <https://github.com/KingFruit85/DND-Randomised-Character-Creator-WIP>
5. **dnd-character-tool** - Character manager (Next.js/TypeScript)
 - URL: <https://github.com/snenenenenenene/dnd-character-tool>

Data Validation

1. **5e-schema** - JSON schema for D&D 5e modules
 - URL: <https://github.com/Swimminschrage/5e-schema>
 - Use case: Validate data before passing to Typst templates

Integration Notes

- **Data Flow:** Tools like statblock5e-creator can generate JSON → validate with 5e-schema → feed into Typst templates
- **Character Data:** Character builders (orcpub, dnd-character-tool) can provide character data for Typst templates
- **Spell Data:** Spell browsers/managers can provide spell data for Typst templates
- **Campaign Data:** Campaign templates can consume structured data from various sources

API Reference

TypstCompiler

```

class TypstCompiler:
    def __init__(
        self,
        timeout: int = 60,
        max_content_size: int = 10 * 1024 * 1024
    ):
        """
        Initialize Typst compiler.

        Args:
            timeout: Compilation timeout in seconds (default: 60)
            max_content_size: Maximum content size in bytes (default: 10 * 1024 * 1024)
        """

    def compile(
        self,
        typst_content: str,
        output_path: Path,
        working_dir: Optional[Path] = None
    ) -> Path:
        """
        Compile Typst content string to PDF.

        Args:
            typst_content: Typst source code as string
            output_path: Where to save the PDF
            working_dir: Working directory (uses temp dir if None)

        Returns:
            Path to generated PDF
        """

    def compile_file(
        self,

```

```

        typ_file: Path,
        output_path: Path
    ) -> Path:
        """
        Compile a Typst file to PDF.

        Args:
            typ_file: Path to .typ file
            output_path: Where to save the PDF

        Returns:
            Path to generated PDF
        """

```

TypstTemplateRegistry

The `TypstTemplateRegistry` provides auto-discovery and management of Typst template wrappers. It automatically scans the `wrappers/` directory and extracts metadata from wrapper modules.

Auto-Discovery

The registry automatically discovers templates by:

1. Scanning `src/waft/templates/typst/wrappers/` for Python modules
2. Looking for functions matching the pattern `generate_*`
3. Extracting metadata from module docstrings
4. Building a searchable index of all templates

Methods

`list_templates()`

```
def list_templates(self) -> List[TypstTemplateMetadata]:
    """
    List all registered templates.

    Returns a list of all templates discovered in the wrappers directory.
    Templates are returned in discovery order (typically alphabetical by name).

    Returns:
        List of TypstTemplateMetadata objects, one for each discovered template.

    Example:
        >>> registry = get_typst_registry()
        >>> templates = registry.list_templates()
        >>> for template in templates:
        ...     print(f"{template.name} ({template.category})")
    """
```

`get_template()`

```
def get_template(self, name: str) -> Optional[TypstTemplateMetadata]:
    """
    Get template by name (case-insensitive).

    Searches for a template matching the given name. The search is case
    and matches against the template's name field.

    Args:
        name: Template name to search for (case-insensitive)

    Returns:
        TypstTemplateMetadata if found, None otherwise

    Example:
        >>> registry = get_typst_registry()
        >>> template = registry.get_template("Flow Way")
        >>> if template:
        ...     print(f"Found: {template.description}")
    """
```

`get_generate_function()`

```
def get_generate_function(self, template_name: str) -> Optional[Callable]:
    """
    Get the generate function for a template.

    Retrieves the actual callable function that generates PDFs for the
    template. This is the function you call to generate documents.

    Args:
        template_name: Name of the template (case-insensitive)

    Returns:
        Callable function if found, None otherwise. The function signature
        by template but typically accepts (title, content, output_path,

    Example:
        >>> registry = get_typst_registry()
        >>> generate = registry.get_generate_function("Flow Way")
        >>> if generate:
        ...     pdf = generate(
        ...         title="My Document",
        ...         content="# Content",
        ...         output_path=Path("output.pdf")
        ...     )
    """
```

`search()`

```
def search(self, query: str) -> List[TypstTemplateMetadata]:
    """
    Search templates by name, description, or tags.

    Performs a case-insensitive search across template names, descriptions,
    and tags. Returns all templates that match the query in any of these fields.

    Args:
        query: Search query string (case-insensitive)

    Returns:
        List of matching TypstTemplateMetadata objects

    Example:
        >>> registry = get_typst_registry()
        >>> results = registry.search("academic")
        >>> print(f"Found {len(results)} academic templates")
    """
```

`get_categories()`

```
def get_categories(self) -> List[str]:
    """
    Get all unique categories.

    Returns a sorted list of all unique categories used by registered templates.
    Useful for filtering or organizing templates by type.

    Returns:
        Sorted list of category strings

    Example:
        >>> registry = get_typst_registry()
        >>> categories = registry.get_categories()
        >>> print(f"Available categories: {' '.join(categories)}")
    """
```

```
get_tags()
```

```
def get_tags(self) -> List[str]:
    """
    Get all unique tags.

    Returns a sorted list of all unique tags used by registered templates.
    Tags are useful for cross-cutting categorization (e.g., "academic",
    "business").

    Returns:
        Sorted list of tag strings

    Example:
        >>> registry = get_typst_registry()
        >>> tags = registry.get_tags()
        >>> print(f"Available tags: {' '.join(tags)}")
    """
```

`count()`

```
def count(self) -> int:
    """
    Get total number of templates.

    Returns the total count of registered templates. Useful for statist
    or validation.

    Returns:
        Integer count of templates

    Example:
        >>> registry = get_typst_registry()
        >>> print(f"Total templates: {registry.count()}")
    """
```

Global Registry Access

```
def get_typst_registry() -> TypstTemplateRegistry:
    """
    Get the global Typst template registry instance.

    Returns a singleton instance of TypstTemplateRegistry. The registry
    is initialized on first access and cached for subsequent calls.

    Returns:
        TypstTemplateRegistry instance

    Example:
        >>> from src.waft.templates.typst import get_typst_registry
        >>> registry = get_typst_registry()
        >>> templates = registry.list_templates()
    """
```

TypstTemplateMetadata

```
@dataclass
class TypstTemplateMetadata:
    name: str
    module_name: str
    description: str
    category: str = "general"
    tags: List[str] = field(default_factory=list)
    generate_function: Optional[str] = None
    parameters: Dict[str, Any] = field(default_factory=dict)
    example_usage: Optional[str] = None
    author: Optional[str] = None
    version: str = "1.0.0"
    status: str = "production"
    source_repo: Optional[str] = None
```

Security Features

The TypstCompiler includes comprehensive security hardening:

Path Validation

- Rejects paths containing `..` (path traversal protection)
- Validates absolute paths are within allowed directories
- Resolves symlinks before validation

Input Validation

- Content size limits (default: 10MB, configurable)
- Compilation timeout (default: 60 seconds, configurable)

Subprocess Security

- **Never uses** `shell=True` - all subprocess calls use `shell=False`
- List-based command arguments
- Explicit timeout handling

File Permissions

- Checks read/write permissions before operations
- Creates directories with safe permissions (0o755)

Error Handling

Common Errors

Missing Typst CLI

```
RuntimeError: Typst CLI not found. Please install Typst:  
- Using Cargo: cargo install typst-cli  
- Or download from: https://typst.com
```

Path Validation Error

```
ValueError: Path traversal detected: ../../../../etc/passwd.  
Paths containing '..' are not allowed for security reasons.
```

Content Size Limit

```
ValueError: Content size (11000000 bytes) exceeds maximum  
allowed size (10485760 bytes).
```

Compilation Timeout

```
RuntimeError: Typst compilation timed out after 60 seconds.  
Consider increasing timeout or simplifying the document.
```

Compilation Failure

```
RuntimeError: Typst compilation failed:  
[Typst error messages]
```

Examples

Example 1: Simple Document

```
from pathlib import Path  
from src.waft.templates.typst import TypstCompiler  
  
compiler = TypstCompiler()  
  
content = """  
#set page(margin: 2cm)  
  
= My Document  
  
== Introduction  
  
This is my document content.  
"""  
  
pdf = compiler.compile(content, Path("simple.pdf"))
```

Example 2: Academic Paper

```
from pathlib import Path
from src.waft.templates.typst import get_typst_registry

registry = get_typst_registry()
generate = registry.get_generate_function("arkheion")

pdf = generate(
    title="Research Findings",
    content="""
= Introduction

Our research shows...

= Methodology

We conducted experiments...

= Results

The results indicate...
""",
    output_path=Path("research.pdf"),
    authors=[
        {
            "name": "Dr. Jane Smith",
            "email": "jane@university.edu",
            "affiliation": "University of Science",
            "orcid": "0000-0000-0000-0000"
        }
    ],
    abstract="This paper presents novel findings...",
    keywords=["research", "science", "findings"],
    date="January 2026",
```

```
        bibliography="references.bib"  
    )
```

Example 3: Business Letter

```
from pathlib import Path  
from src.waft.templates.typst import get_typst_registry  
  
registry = get_typst_registry()  
generate = registry.get_generate_function("Appreciated Letter")  
  
pdf = generate(  
    content="""  
Dear Mr. Johnson,  
  
I am writing to follow up on our previous conversation...  
  
Thank you for your time.  
  
Best regards,  
""",  
    output_path=Path("letter.pdf"),  
    sender="Jane Doe\nABC Company\n123 Business St\nCity, ST 12345",  
    recipient="Mr. John Johnson\nXYZ Corporation\n456 Corporate Ave\nCity, ST 12345",  
    date="January 19, 2026",  
    subject="Follow-up on Proposal",  
    name="Jane Doe\nSenior Manager"  
)
```

Example 4: Search Templates

```
from src.waft.templates.typst import get_typst_registry

registry = get_typst_registry()

# Search by category
papers = [t for t in registry.list_templates() if t.category == "paper"]
print(f"Found {len(papers)} paper templates")

# Search by keyword
results = registry.search("academic")
for template in results:
    print(f"- {template.name}: {template.description[:50]}...")

# Get templates by tag
all_templates = registry.list_templates()
academic_templates = [
    t for t in all_templates
    if "academic" in t.tags or "paper" in t.tags
]
```

Example 5: Batch Processing

Generate multiple PDFs efficiently:

```

from pathlib import Path
from src.waft.templates.typst import TypstCompiler, get_typst_registry

compiler = TypstCompiler()
registry = get_typst_registry()
generate = registry.get_generate_function("Flow Way")

# Process multiple documents
documents = [
    {"title": "Report 1", "content": "# Report 1\n\nContent..."},
    {"title": "Report 2", "content": "# Report 2\n\nContent..."},
    {"title": "Report 3", "content": "# Report 3\n\nContent..."},
]

output_dir = Path("output")
output_dir.mkdir(exist_ok=True)

for i, doc in enumerate(documents, 1):
    pdf_path = generate(
        title=doc["title"],
        content=doc["content"],
        output_path=output_dir / f"report_{i}.pdf",
        authors=["WAFT Team"],
        year=2026
    )
    print(f"Generated: {pdf_path}")

```

Example 6: Dynamic Content Generation

Generate documents from structured data:

```

from pathlib import Path
from src.waft.templates.typst import TypstCompiler

compiler = TypstCompiler()

# Generate content from data
def generate_report(data):
    sections = "\n\n".join([
        f"== {section['title']}\n\n{section['content']}"
        for section in data['sections']
    ])

    typst_content = f"""
#set page(margin: 2.5cm)
#set text(font: "Linux Libertine", size: 11pt)

= {data['title']}

#align(center)[
    *{data['subtitle']}*

    {data['date']}
]

{sections}

== References

#bibliography("references.bib")
"""

    return compiler.compile(
        typst_content=typst_content,
        output_path=Path(f"{data['title'].lower().replace(' ', '_')}.pdf")
    )

```

```
)

# Use with structured data
report_data = {
    "title": "Quarterly Report",
    "subtitle": "Q1 2026",
    "date": "January 2026",
    "sections": [
        {"title": "Executive Summary", "content": "Summary text..."},
        {"title": "Financials", "content": "Financial data..."},
        {"title": "Conclusion", "content": "Conclusion text..."},
    ]
}

pdf = generate_report(report_data)
```

Example 7: Error Handling

Robust error handling for production use:

```

from pathlib import Path
from src.waft.templates.typst import TypstCompiler
import logging

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

compiler = TypstCompiler()

def safe_compile(content: str, output_path: Path) -> bool:
    """Safely compile Typst content with comprehensive error handling."""
    try:
        pdf_path = compiler.compile(
            typst_content=content,
            output_path=output_path
        )
        logger.info(f"Successfully generated: {pdf_path}")
        return True

    except RuntimeError as e:
        if "not found" in str(e).lower():
            logger.error("Typst CLI not installed. Please install Typst")
        elif "timed out" in str(e).lower():
            logger.error(f"Compilation timed out. Content may be too co
        else:
            logger.error(f"Compilation failed: {e}")
        return False

    except ValueError as e:
        if "path traversal" in str(e).lower():
            logger.error(f"Security error: {e}")
        elif "size" in str(e).lower():
            logger.error(f"Content too large: {e}")
        else:

```

```
        logger.error(f"Validation error: {e}")
        return False

    except Exception as e:
        logger.error(f"Unexpected error: {e}", exc_info=True)
        return False

# Use with error handling
success = safe_compile(
    content="# My Document\n\nContent...",
    output_path=Path("output.pdf")
)

if success:
    print("Document generated successfully!")
else:
    print("Failed to generate document. Check logs for details.")
```

Example 8: Working with Images and Assets

Include images and other assets in your documents:

```

from pathlib import Path
from src.waft.templates.typst import TypstCompiler

compiler = TypstCompiler()

# Ensure images are accessible
image_path = Path("assets/logo.png")
if not image_path.exists():
    raise FileNotFoundError(f"Image not found: {image_path}")

typst_content = f"""
#set page(margin: 2cm)

= Document with Images

#align(center)[
    #image("{image_path}", width: 5cm)
]

== Introduction

This document includes images and other assets.

#figure(
    image("{image_path}", width: 10cm),
    caption: [Company Logo]
)
"""

# Compile with working directory containing assets
pdf_path = compiler.compile(
    typst_content=typst_content,
    output_path=Path("document_with_images.pdf"),

```

```
        working_dir=Path(".")    # Use current directory for asset resolution
    )
```

Example 9: Using Typst Packages

Leverage packages from Typst Universe:

```
from pathlib import Path
from src.waft.templates.typst import TypstCompiler

compiler = TypstCompiler()

typst_content = """
#import "@preview/codly:0.2.1": code-block, code

= Code Documentation

This document uses the `codly` package for syntax highlighting.

#code-block("python", [
def hello_world():
    print("Hello, Typst!")
])

Inline code: `#code("python", "print('Hello')")`
"""

pdf_path = compiler.compile(
    typst_content=typst_content,
    output_path=Path("code_documentation.pdf")
)
```

Example 10: Multi-Page Documents with Complex Layouts

Create complex multi-page documents:

```
from pathlib import Path
from src.waft.templates.typst import TypstCompiler

compiler = TypstCompiler()

typst_content = """
#set page(
    paper: "a4",
    margin: (top: 2.5cm, bottom: 2cm, left: 2cm, right: 2cm)
)
#set text(font: "Linux Libertine", size: 11pt)
#set heading(numbering: "1.")

= Main Title

== First Section

Content for the first section goes here.

== Second Section

#lorem(50)

== Third Section

#grid(
    columns: 2,
    gutter: 1cm,
    [First column content],
    [Second column content]
)

== Tables
```

```
#table(  
    columns: 3,  
    [*Header 1*], [*Header 2*], [*Header 3*],  
    [Cell 1], [Cell 2], [Cell 3],  
    [Cell 4], [Cell 5], [Cell 6],  
)  
"""  
  
pdf_path = compiler.compile(  
    typst_content=typst_content,  
    output_path=Path("complex_document.pdf")  
)
```

Creating Custom Templates

To add a new template wrapper:

1. Create a wrapper module in `src/waft/templates/typst/wrappers/`
2. Implement a `generate_*` function
3. Add module docstring with metadata

Example:

```

"""
My Custom Template Wrapper
=====

Description of the template.

Category: report
Tags: [typst, custom, report]
Source: my-template
"""

from pathlib import Path
from ..compiler import TypstCompiler

def generate_my_template(
    title: str,
    content: str,
    output_path: Path,
    **kwargs
) -> Path:
    """
    Generate PDF using My Custom Template.

    Args:
        title: Document title
        content: Main content
        output_path: Output PDF path
        **kwargs: Additional parameters

    Returns:
        Path to generated PDF
    """
    typst_content = f'''
#import "@preview/my-template:1.0.0": template

```

```
#show: template.with(title: "{title}")

{content}

'''

    compiler = TypstCompiler()
    return compiler.compile(typst_content, output_path)
```

The registry will automatically discover and register your template!

Integration with WAFT Systems

DocumentBuilder Integration

Typst templates integrate with WAFT's DocumentBuilder system:

```
from src.waft.templates.typst import get_typst_registry
from src.waft.core.document_builder import DocumentBuilder

registry = get_typst_registry()
generate = registry.get_generate_function("Flow Way")

builder = DocumentBuilder()
builder.add_template("typst", generate)
pdf = builder.build(...)
```

D&D 5e Integration

Typst D&D templates complement existing WAFT D&D functionality:

- **HTML Templates:** `dnd_scenario.py` (interactive scenarios)
- **LaTeX Templates:** `dnd5e_latex.py` (print-ready content)

- **Typst Templates:** `typst-dnd5e`, `wenyuan-campaign` (modern typesetting)

All three systems can consume the same D&D 5e data structures.

Data Pipeline Example

```
# 1. Generate data (external tool or WAFT)
statblock_data = {
    "name": "Dragon",
    "type": "dragon",
    "armor_class": 20,
    # ... more fields
}

# 2. Validate with 5e-schema (optional)
# validate_statblock(statblock_data)

# 3. Generate PDF with Typst template
generate = registry.get_generate_function("dnd5e")
pdf = generate(
    content=format_statblock(statblock_data),
    output_path=Path("dragon_statblock.pdf")
)
```

Troubleshooting

Typst CLI Not Found

Problem: `RuntimeError: Typst CLI not found. Please install Typst: ...`

Symptoms: - Error on `TypstCompiler()` initialization - Typst command not found in PATH

Solutions:

1. **Install Typst:** ```bash # Using Cargo (recommended) cargo install typst-cli

Or download from <https://typst.com> # Extract and add to PATH ```

1. **Verify Installation:** `bash typst --version` # Should output version number

2. **Check PATH:** ```bash # Linux/macOS echo \$PATH which typst

Windows echo %PATH% where typst ```

1. **Restart Terminal:** Close and reopen terminal after installation

Version Too Old

Problem:

```
RuntimeError: Typst version X.X.X is too old. Minimum  
required: 0.10.0
```

Symptoms: - Error on `TypstCompiler()` initialization - Installed Typst version is below 0.10.0

Solutions:

1. **Update Typst:** ```bash # Using Cargo cargo install --force typst-cli

Or download latest from <https://typst.com> ```

1. **Verify Version:** `bash typst --version` # Should be `>= 0.10.0`

Path Validation Errors

Problem: `ValueError: Path traversal detected: ...` or

```
ValueError: Absolute path outside allowed directories
```

Symptoms: - Error when specifying output paths - Paths with `..` are rejected - Absolute paths outside project are rejected

Solutions:

1. **Use Relative Paths:** Use paths relative to project directory ``python #

✓ Good `compiler.compile(content, Path("output/document.pdf"))`

✗ Bad `compiler.compile(content, Path("../../etc/passwd")) ```

1. **Use Allowed Directories:** Output paths must be within:

2. Project directory

3. Temporary directories (`/tmp` , `/var/tmp` , or system temp dir)

4. **Create Output Directory First:**

```
python output_path =  
Path("output/document.pdf")  
output_path.parent.mkdir(parents=True, exist_ok=True)  
compiler.compile(content, output_path)
```

Compilation Timeout

Problem: `RuntimeError: Typst compilation timed out after 60 seconds`

Symptoms: - Compilation takes too long - Timeout error for complex documents

Solutions:

1. **Increase Timeout:**

```
python # For large documents compiler =  
TypstCompiler(timeout=300) # 5 minutes pdf =  
compiler.compile(large_content, output_path)
```

2. **Simplify Document:** Break complex documents into smaller parts

3. **Check for Infinite Loops:** Review Typst code for potential infinite loops

4. **Optimize Images:** Reduce image sizes if document contains many images

Content Size Limit

Problem: `ValueError: Content size (11000000 bytes) exceeds maximum allowed size (10485760 bytes)`

Symptoms: - Error when compiling large documents - Content exceeds default 10MB limit

Solutions:

1. **Increase Size Limit** (for trusted content):

```
python compiler = TypstCompiler(max_content_size=50 * 1024 * 1024) # 50MB pdf = compiler.compile(large_content, output_path)
```
2. **Split Content:** Break large documents into multiple smaller documents
3. **External Assets:** Move large content to external files and reference them

Compilation Failures

Problem: `RuntimeError: Typst compilation failed: [error messages]`

Symptoms: - PDF not generated - Typst syntax errors in content

Solutions:

1. **Check Typst Syntax:** Validate Typst code syntax

```
bash typst compile document.typ # Check for syntax errors
```
2. **Review Error Messages:** Typst provides detailed error messages with line numbers
3. **Validate Content:** Test Typst content in Typst web editor first
4. **Check Dependencies:** Ensure all imported packages are available
5. **Common Issues:**
 6. Missing closing braces or brackets
 7. Invalid function calls

8. Missing required parameters

9. Package import errors

Template Not Found

Problem: Template not appearing in registry or `get_template()` returns None

Symptoms: - Template not in `list_templates()` - `get_template()` returns None - `get_generate_function()` returns None

Solutions:

1. **Check File Exists:** Verify wrapper file exists in `wrappers/` directory

```
bash ls src/waft/templates/typst/wrappers/
```

2. **Verify Function Name:** Function must match pattern `generate_*`

```
```python # ✅ Good def generate_my_template(...): ...
```

```
❌ Bad def my_template_generator(...): ... ```
```

1. **Check Module Docstring:** Module must have docstring with metadata

```
```python """ Template Name =====
```

Description here.

```
Category: report Tags: [typst, report] """ ```
```

1. **Check for Import Errors:** Look for import errors in console/logs

```
python import logging
```

```
logging.basicConfig(level=logging.WARNING) # Check for  
warnings about failed imports
```

2. **Reload Registry:** Registry is cached - restart Python process to reload

Permission Errors

Problem: `PermissionError: [Errno 13] Permission denied`

Symptoms: - Cannot write to output directory - Cannot read input files

Solutions:

1. **Check Write Permissions:**

```
python output_path = Path("output/document.pdf")  
output_path.parent.mkdir(parents=True, exist_ok=True) #  
Ensure directory is writable
```
2. **Check File Permissions:**

```
bash ls -l document.typ # Check  
read permissions ls -ld output/ # Check write  
permissions
```
3. **Fix Permissions:**

```
bash chmod 644 document.typ # Make  
readable chmod 755 output/ # Make writable
```

Performance Issues

Problem: Slow compilation or high memory usage

Symptoms: - Compilation takes longer than expected - High memory consumption

Solutions:

1. **Optimize Typst Code:** Use efficient Typst patterns
2. **Reduce Image Sizes:** Compress images before including
3. **Cache Results:** Cache compiled PDFs for repeated content
4. **Batch Processing:** Process multiple documents in parallel (with caution)

Package Import Errors

Problem: Typst package not found or import fails

Symptoms: - Error: `Package not found: @preview/package-name` - Import errors in Typst content

Solutions:

1. **Check Package Name:** Verify package name and version
2. **Update Typst:** Ensure Typst version supports the package

3. **Check Typst Universe:** Verify package exists at <https://typst.app/universe/>
4. **Local Packages:** For local packages, ensure paths are correct

Still Having Issues?

If you're still experiencing problems:

1. **Check Logs:** Enable detailed logging to see what's happening `python`

```
import logging
```

```
logging.basicConfig(level=logging.DEBUG)
```

2. **Minimal Example:** Create a minimal example to isolate the issue

```
```python from pathlib import Path from src.waft.templates.typst import  
TypstCompiler
```

```
compiler = TypstCompiler() pdf = compiler.compile("# Hello", Path("test.pdf"))
```
```

1. **Typst CLI Directly:** Test with Typst CLI directly `bash echo "#`

```
Hello" > test.typ typst compile test.typ test.pdf
```

2. **Open an Issue:** If problem persists, open an issue with:

3. Error message
4. Typst version (`typst --version`)
5. Python version
6. Minimal reproducible example

Testing

Run the test suite:

```
# Run all Typst tests
uv run pytest tests/test_typst_compiler.py tests/test_typst_registry.py

# Run specific test
uv run pytest tests/test_typst_compiler.py::TestBasicCompilation -v
```

Architecture

```
src/waft/templates/typst/
├─ __init__.py          # Module exports
├─ compiler.py          # TypstCompiler class
├─ registry.py          # TypstTemplateRegistry class
├─ wrappers/            # Template wrapper modules
│   ├─ __init__.py
│   ├─ arkheion.py
│   ├─ flow_way.py
│   ├─ appreciated_letter.py
│   ├─ charged_ieee.py
│   ├─ unequivocal_ams.py
│   ├─ wondrous_book.py
│   ├─ dashing_dept_news.py
│   ├─ badformer.py
│   ├─ cereal_words.py
│   └─ icicle.py
└─ templates/           # Local template files (optional)
    └─ flow-way/        # Cloned template repositories
```

Contributing

When adding new templates:

1. Follow the wrapper pattern in existing templates
2. Include comprehensive docstrings
3. Add appropriate category and tags
4. Test compilation with sample content
5. Update this README with usage examples

License

This infrastructure is part of WAFT and follows the project's license.

References

- [Typst Documentation](#)
- [Typst Templates Repository](#)
- [Typst Universe](#)

Migration from LaTeX

If you're migrating from LaTeX to Typst, here are key differences and migration tips:

Key Differences

1. **Syntax:** Typst uses `#` for commands instead of `\` ````typst // Typst =`
Heading `#set text(size: 12pt)`

`// LaTeX \section{Heading} \fontsize{12pt}{14pt}\selectfont ````

1. **Compilation:** Single-pass vs. multi-pass
2. Typst: Single pass (faster)

3. LaTeX: Multiple passes (for references, TOC, etc.)

4. **Packages:** Different package system

5. Typst: `#import "@preview/package:version": item`

6. LaTeX: `\usepackage{package}`

Migration Guide

1. **Start Simple:** Migrate simple documents first
2. **Use Templates:** Leverage existing Typst templates
3. **Test Incrementally:** Test each section as you migrate
4. **Check Compatibility:** Some LaTeX features may not have direct Typst equivalents

Common Migrations

Document Class → Page Setup:

```
// Typst
#set page(paper: "a4", margin: 2.5cm)

// LaTeX
\documentclass[a4paper]{article}
\usepackage[margin=2.5cm]{geometry}
```

Sections:

```
// Typst
= Section
== Subsection
=== Subsubsection

// LaTeX
\section{Section}
\subsection{Subsection}
\subsubsection{Subsubsection}
```

Math:

```
// Typst
$ E = mc^2 $
$ \sum_{n=1}^{\infty} 1/n^2 = \pi^2/6 $

// LaTeX
$E = mc^2$
$\sum_{n=1}^{\infty} \frac{1}{n^2} = \frac{\pi^2}{6}$
```

Tips and Tricks

Productivity Tips

1. **Use Typst Web Editor:** Test Typst code in the web editor before compiling
2. **Leverage Auto-completion:** Use IDE plugins for Typst syntax highlighting
3. **Template Library:** Build a library of reusable template functions
4. **Version Control:** Track Typst source files in version control, not just PDFs

Common Patterns

Reusable Components:

```
def create_table(headers: List[str], rows: List[List[str]]) -> str:
    header_row = ", ".join(f"[*{h}*)" for h in headers)
    data_rows = "\n".join(
        ", ".join(f"[{cell}]" for cell in row)
        for row in rows
    )
    return f"#table(columns: {len(headers)},\n{header_row},\n{data_rows})"
```

Conditional Sections:

```
def generate_with_optional_sections(data: Dict) -> str:
    sections = []
    if data.get('abstract'):
        sections.append(f"== Abstract\n\n{data['abstract']}")
    if data.get('introduction'):
        sections.append(f"== Introduction\n\n{data['introduction']}")
    return "\n\n".join(sections)
```

Dynamic Styling:

```
def generate_with_theme(theme: str, content: str) -> str:
    themes = {
        'dark': '#set text(fill: white)\n#set page(fill: rgb("1a1a1a"))',
        'light': '#set text(fill: black)\n#set page(fill: white)',
    }
    return f"{themes.get(theme, themes['light'])}\n\n{content}"
```

Debugging Tips

1. **Start Small:** Test with minimal content first

2. **Check Syntax:** Use Typst web editor to validate syntax
3. **Read Errors:** Typst provides detailed error messages with line numbers
4. **Isolate Issues:** Comment out sections to isolate problems
5. **Use Logging:** Enable debug logging to see what's happening

Performance Tips

1. **Optimize Images:** Compress and resize images before including
2. **Avoid Redundancy:** Don't repeat the same content multiple times
3. **Use Efficient Functions:** Prefer Typst's built-in functions
4. **Cache When Possible:** Cache compiled PDFs for repeated content

Support

For issues or questions:

1. **Check Documentation:** Review this README and Typst documentation
2. **Troubleshooting:** Check the troubleshooting section above
3. **Template Issues:** Review template-specific documentation
4. **Typst Issues:** Check [Typst documentation](#) for Typst-specific issues
5. **WAFT Issues:** Open an issue in the WAFT repository with:
 6. Error message and stack trace
 7. Typst version (`typst --version`)
 8. Python version
 9. Minimal reproducible example
 10. Expected vs. actual behavior

Additional Resources

Typst Resources

- **Official Documentation:** <https://typst.app/docs/>
- **Typst Universe:** <https://typst.app/universe/> (package repository)
- **Typst Web Editor:** <https://typst.app/> (online editor)

- **Typst Templates:** <https://github.com/typst/templates>
- **Typst Discord:** Community support and discussions

WAFT Resources

- **WAFT Documentation:** See main WAFT documentation
- **LaTeX Templates:** See `src/waft/templates/latex/` for LaTeX template examples
- **Template Patterns:** Review existing template wrappers for patterns

Learning Typst

- **Tutorial:** Start with Typst's official tutorial
- **Examples:** Browse Typst Universe for example packages
- **Practice:** Use Typst web editor to experiment
- **Community:** Join Typst Discord for help and discussions